Java OOP Study Notes: Interface

1. Definition & Purpose

An **interface** in Java is a reference type that defines a contract — a set of abstract methods that implementing classes must provide. It specifies WHAT a class must do, not HOW it does it.

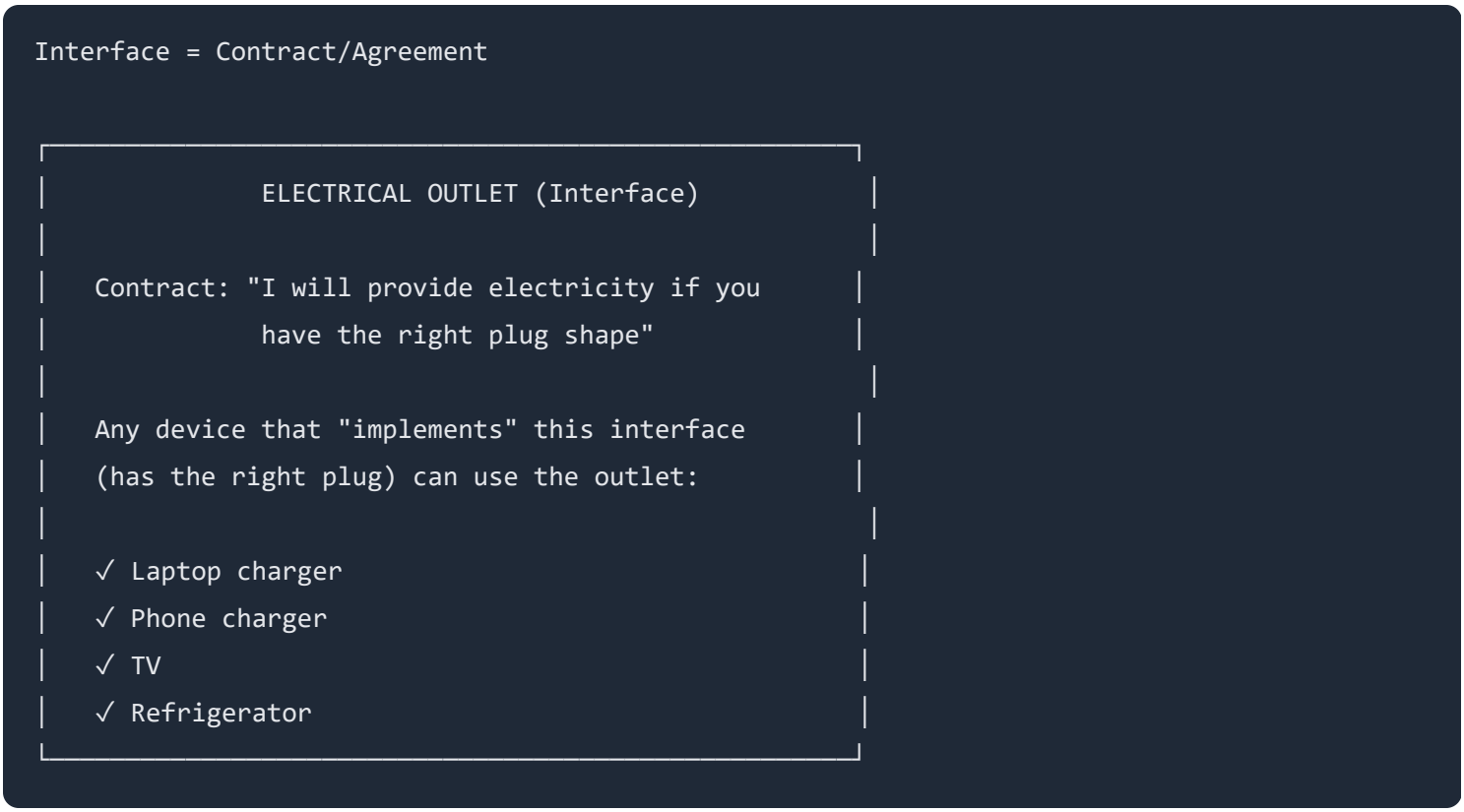
Syntax

```
interface InterfaceName {
    // Constants
    // Abstract methods
    // Default methods (Java 8+)
    // Static methods (Java 8+)
    // Private methods (Java 9+)
}
```

Purpose of Interfaces

Purpose	Description
Abstraction	Hide implementation, show only functionality
Contract	Define what methods a class must implement
Multiple Inheritance	A class can implement multiple interfaces
Loose Coupling	Program to interface, not implementation
Polymorphism	Treat different objects uniformly

Real-World Analogy



Basic Example

```
// Interface definition
interface Drawable {
    void draw();  // Abstract method - no body
}

// Class implementing the interface
class Circle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}

class Rectangle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a rectangle");
    }
}

// Usage
Drawable d1 = new Circle();
Drawable d2 = new Rectangle();

d1.draw();  // Drawing a circle
d2.draw();  // Drawing a rectangle
```

---

2. Implementing Interface: `implements` Keyword

A class uses the `implements` keyword to implement an interface. The class MUST provide implementations for ALL abstract methods in the interface.

Syntax

```
class ClassName implements InterfaceName {
    // Must implement all abstract methods
}
```

Rules for Implementation

Rule	Description
Must implement ALL abstract methods	Or declare class as `abstract`
Implemented methods must be `public`	Cannot reduce visibility
Use `@Override` annotation	Recommended for clarity
Can implement multiple interfaces	Separated by comma

Single Interface Implementation

```
interface Printable {
    void print();
    int getPageCount();
}

class Document implements Printable {
    private String content;
    private int pages;

    Document(String content, int pages) {
        this.content = content;
        this.pages = pages;
    }

    @Override
    public void print() {
        System.out.println("Printing: " + content);
    }

    @Override
    public int getPageCount() {
        return pages;
    }
}

// Usage
Printable doc = new Document("Hello World", 5);
doc.print();
System.out.println("Pages: " + doc.getPageCount());
```

Multiple Interface Implementation

```
interface Drawable {
    void draw();
}

interface Colorable {
    void setColor(String color);
    String getColor();
}

interface Resizable {
    void resize(double factor);
}

// Implementing multiple interfaces
class Shape implements Drawable, Colorable, Resizable {
    private String color;
    private double size;

    Shape(double size) {
        this.size = size;
        this.color = "Black";
    }

    @Override
    public void draw() {
        System.out.println("Drawing shape of size " + size);
    }

    @Override
    public void setColor(String color) {
        this.color = color;
    }

    @Override
    public String getColor() {
        return color;
    }
```

```
    @Override
    public void resize(double factor) {
        size *= factor;
        System.out.println("Resized to: " + size);
    }
}
```

Partial Implementation (Abstract Class)

If a class doesn't implement ALL methods, it must be declared `abstract`:

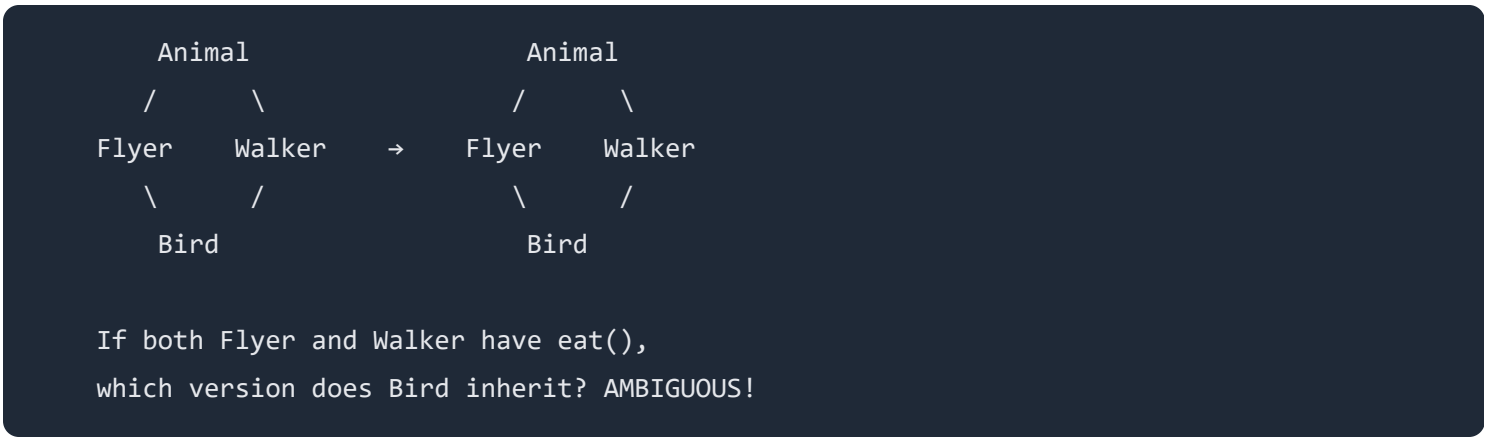
```
interface Animal {
    void eat();
    void sleep();
    void makeSound();
}

// Partial implementation - must be abstract
abstract class Pet implements Animal {
    @Override
    public void eat() {
        System.out.println("Pet eating");
    }

    @Override
    public void sleep() {
        System.out.println("Pet sleeping");
    }

    // makeSound() not implemented - left for subclasses
}

// Complete implementation
class Dog extends Pet {
    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }
}
```

---

3. Multiple Inheritance via Interface

Java doesn't support multiple inheritance with classes (to avoid the Diamond Problem), but DOES support it with interfaces.

The Problem with Class Multiple Inheritance



The Solution: Interfaces

```
// Multiple interfaces - no ambiguity
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

interface Walkable {
    void walk();
}

// A class can implement ANY number of interfaces
class Duck implements Flyable, Swimmable, Walkable {

    @Override
    public void fly() {
        System.out.println("Duck is flying");
    }

    @Override
    public void swim() {
        System.out.println("Duck is swimming");
    }

    @Override
    public void walk() {
        System.out.println("Duck is walking");
    }
}

class Penguin implements Swimmable, Walkable {
    // Penguin can't fly!

    @Override
    public void swim() {
        System.out.println("Penguin swimming gracefully");
```

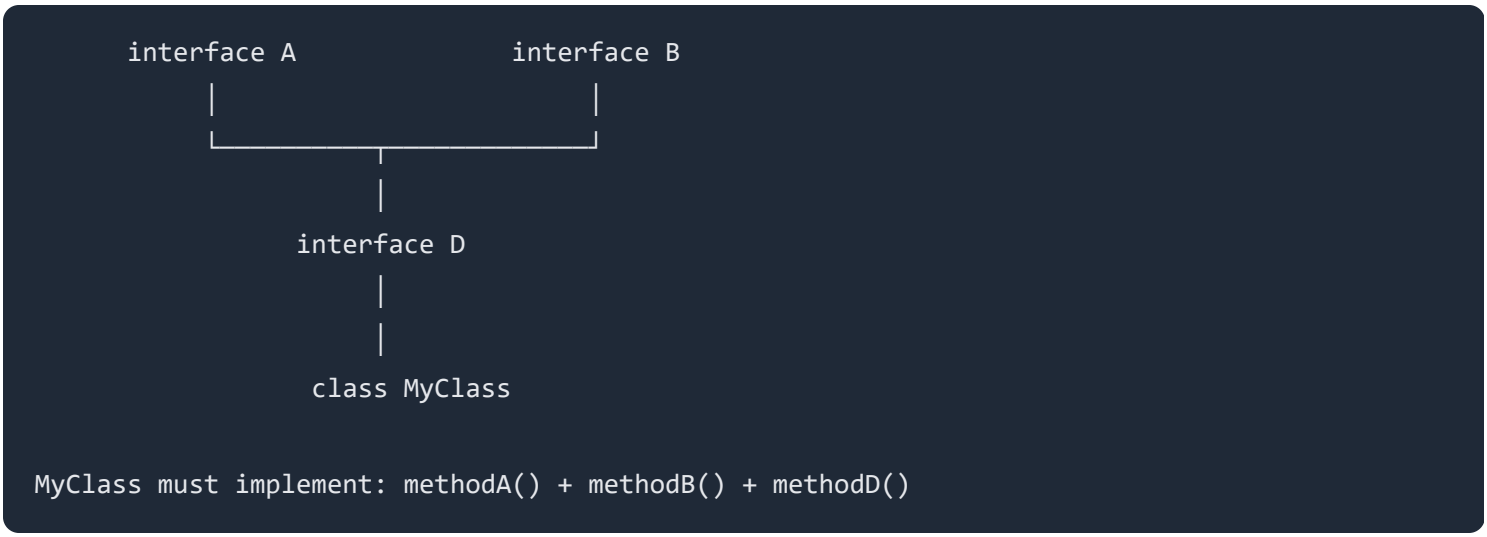
```
    }

    @Override
    public void walk() {
        System.out.println("Penguin waddling");
    }
}

class Eagle implements Flyable, Walkable {
    // Eagle can't swim well

    @Override
    public void fly() {
        System.out.println("Eagle soaring high");
    }

    @Override
    public void walk() {
        System.out.println("Eagle walking");
    }
}
```

Interface Inheritance

Interfaces can extend other interfaces:

```
interface A {
    void methodA();
}

interface B {
    void methodB();
}

// Interface extending single interface
interface C extends A {
    void methodC();
}

// Interface extending MULTIPLE interfaces!
interface D extends A, B {
    void methodD();
}

// Class implementing D must implement ALL methods
class MyClass implements D {
    @Override
    public void methodA() { }  // From A

    @Override
    public void methodB() { }  // From B

    @Override
    public void methodD() { }  // From D
}
```

Inheritance Hierarchy



Combining Class Extension with Interface Implementation

```
class Animal {
    void breathe() {
        System.out.println("Breathing...");
    }
}

interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

// Extends ONE class, implements MULTIPLE interfaces
class Duck extends Animal implements Flyable, Swimmable {

    @Override
    public void fly() {
        System.out.println("Duck flying");
    }

    @Override
    public void swim() {
        System.out.println("Duck swimming");
    }
}

// Usage
Duck duck = new Duck();
duck.breathe();  // From Animal class
duck.fly();      // From Flyable interface
duck.swim();     // From Swimmable interface
```

4. Variables in Interface: `public static final`

All variables declared in an interface are implicitly `public`, `static`, and `final` — they are constants.

Syntax

```
interface MyInterface {
    // These are ALL equivalent:
    int VALUE1 = 100;
    public int VALUE2 = 100;
    static int VALUE3 = 100;
    final int VALUE4 = 100;
    public static final int VALUE5 = 100;  // Explicit (recommended for clarity)
}
```

Rules for Interface Variables

Rule	Description
Always `public`	Cannot be private or protected
Always `static`	Belongs to interface, not instance
Always `final`	Cannot be changed (constant)
Must be initialized	No uninitialized variables

Example

```
interface MathConstants {
    double PI = 3.14159265359;
    double E = 2.71828182846;
    int MAX_ITERATIONS = 1000;
}

interface HttpStatus {
    int OK = 200;
    int CREATED = 201;
    int BAD_REQUEST = 400;
    int UNAUTHORIZED = 401;
    int NOT_FOUND = 404;
    int SERVER_ERROR = 500;
}

interface Config {
    String APP_NAME = "MyApplication";
    String VERSION = "1.0.0";
    int MAX_USERS = 100;
    boolean DEBUG_MODE = false;
}

public class Main {
    public static void main(String[] args) {
        // Access using interface name
        System.out.println("PI = " + MathConstants.PI);
        System.out.println("Status OK = " + HttpStatus.OK);
        System.out.println("App: " + Config.APP_NAME);

        // Cannot modify - they are final
        // MathConstants.PI = 3.14;  // ERROR! Cannot assign to final variable
    }
}
```

Constants vs Enum

For related constants, consider using `enum` instead:

```
// Using interface (old style)
interface Status {
    int PENDING = 0;
    int APPROVED = 1;
    int REJECTED = 2;
}

// Using enum (better for related constants)
enum Status {
    PENDING, APPROVED, REJECTED
}
```

---

5. Methods in Interface

5.1 Abstract Methods (Default is `public abstract`)

All regular methods in an interface are implicitly `public` and `abstract`.

```
interface Drawable {
    // These are ALL equivalent:
    void draw();
    public void draw();
    abstract void draw();
    public abstract void draw();  // Explicit (optional)
}
```

5.2 Default Methods (Java 8+)

Default methods have a body and provide default implementation. Implementing classes can use or override them.

```
interface Vehicle {
    void start();
    void stop();

    // Default method - has implementation
    default void honk() {
        System.out.println("Beep! Beep!");
    }

    default void displayInfo() {
        System.out.println("This is a vehicle");
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car starting...");
    }

    @Override
    public void stop() {
        System.out.println("Car stopping...");
    }

    // Uses default honk() as-is

    // Overrides default displayInfo()
    @Override
    public void displayInfo() {
        System.out.println("This is a car");
    }
}

class Bike implements Vehicle {
    @Override
    public void start() {
        System.out.println("Bike starting...");
    }
```

```
    @Override
    public void stop() {
        System.out.println("Bike stopping...");
    }

    // Override honk with bike-specific sound
    @Override
    public void honk() {
        System.out.println("Ring ring!");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle car = new Car();
        Vehicle bike = new Bike();

        car.honk();   // Beep! Beep! (default)
        bike.honk();  // Ring ring! (overridden)

        car.displayInfo();   // This is a car (overridden)
        bike.displayInfo();  // This is a vehicle (default)
    }
}
```

Resolving Default Method Conflicts

When a class implements multiple interfaces with the same default method:

```
interface A {
    default void greet() {
        System.out.println("Hello from A");
    }
}

interface B {
    default void greet() {
        System.out.println("Hello from B");
    }
}

class C implements A, B {
    // MUST override to resolve conflict
    @Override
    public void greet() {
        // Option 1: Own implementation
        System.out.println("Hello from C");

        // Option 2: Call specific interface's method
        A.super.greet();  // Calls A's version
        B.super.greet();  // Calls B's version
    }
}
```

5.3 Static Methods (Java 8+)

Static methods belong to the interface itself, not implementing classes. They cannot be overridden.

```
interface StringUtils {
    // Static methods - utility functions
    static boolean isEmpty(String str) {
        return str == null || str.trim().isEmpty();
    }

    static String capitalize(String str) {
        if (isEmpty(str)) return str;
        return str.substring(0, 1).toUpperCase() + str.substring(1).toLowerCase();
    }

    static String reverse(String str) {
        if (isEmpty(str)) return str;
        return new StringBuilder(str).reverse().toString();
    }

    static int countWords(String str) {
        if (isEmpty(str)) return 0;
        return str.trim().split("\\s+").length;
    }
}

public class Main {
    public static void main(String[] args) {
        // Call using interface name (NOT through implementing class)
        System.out.println(StringUtils.isEmpty(""));         // true
        System.out.println(StringUtils.capitalize("jAVA"));  // Java
        System.out.println(StringUtils.reverse("Hello"));    // olleH
        System.out.println(StringUtils.countWords("Hello World")); // 2
    }
}
```

Static Methods Are NOT Inherited

```
interface A {
    static void show() {
        System.out.println("Static in A");
    }
}

class B implements A {
    // Does NOT inherit show()
}

public class Main {
    public static void main(String[] args) {
        A.show();    // OK - Call on interface
        // B.show(); // ERROR! B doesn't have show()
    }
}
```

5.4 Private Methods (Java 9+)

Private methods are helper methods used by default/static methods within the interface. They cannot be accessed outside the interface.

```
interface Logger {
    // Public abstract method
    void log(String message);

    // Default methods using private helper
    default void logInfo(String message) {
        logWithLevel("INFO", message);
    }

    default void logWarning(String message) {
        logWithLevel("WARNING", message);
    }

    default void logError(String message) {
        logWithLevel("ERROR", message);
    }

    // Private helper method - not visible outside interface
    private void logWithLevel(String level, String message) {
        String timestamp = java.time.LocalDateTime.now().toString();
        System.out.println("[" + timestamp + "] [" + level + "] " + message);
    }

    // Private static helper
    private static String formatMessage(String msg) {
        return msg.trim().toUpperCase();
    }
}

class ConsoleLogger implements Logger {
    @Override
    public void log(String message) {
        System.out.println("LOG: " + message);
    }

    // Cannot access logWithLevel() - it's private to interface
}

public class Main {
    public static void main(String[] args) {
```

```
        Logger logger = new ConsoleLogger();

        logger.log("Basic log");
        logger.logInfo("This is info");
        logger.logWarning("This is warning");
        logger.logError("This is error");
    }
}
```

Summary: Method Types in Interface

Method Type	Keyword	Body	Inherited	Override	Since
Abstract	(none)	No	Yes	Must	Java 1.0
Default	`default`	Yes	Yes	Can	Java 8
Static	`static`	Yes	No	Cannot	Java 8
Private	`private`	Yes	No	Cannot	Java 9
Private Static	`private static`	Yes	No	Cannot	Java 9

---

6. Interface vs Abstract Class

Comparison Table

Feature	Interface	Abstract Class
Keyword	`interface`	`abstract class`
Implementation	`implements`	`extends`
Multiple Inheritance	Yes (can implement many)	No (single extends)
Variables	Only `public static final`	Any type
Constructors	No	Yes
Abstract Methods	Yes (implicit)	Yes (explicit)
Concrete Methods	Default/Static only	Any method
Access Modifiers	Public only	Any (public, protected, private)
Instance Variables	No	Yes

Visual Comparison

When to Use Which?

```
╔═══════════════════════════════════════════════════════════╗
║                    USE INTERFACE WHEN:                    ║
╠═══════════════════════════════════════════════════════════╣
║  • Defining a contract (what methods must exist)          ║
║  • Unrelated classes need same functionality              ║
║  • You need multiple inheritance                          ║
║  • Defining capabilities (Runnable, Comparable, Serializable) ║
║  • API design (loose coupling)                            ║
║                                                           ║
║  Example: Drawable, Printable, Clickable, Sortable        ║
╚═══════════════════════════════════════════════════════════╝


╔═══════════════════════════════════════════════════════════╗
║                 USE ABSTRACT CLASS WHEN:                  ║
╠═══════════════════════════════════════════════════════════╣
║  • Sharing code among closely related classes             ║
║  • Need common state (instance variables)                 ║
║  • Need constructors for initialization                   ║
║  • Need non-public methods                                ║
║  • There's a clear IS-A hierarchy                         ║
║                                                           ║
║  Example: Animal → Dog/Cat, Shape → Circle/Rectangle      ║
╚═══════════════════════════════════════════════════════════╝
```

Code Comparison

```
// INTERFACE approach
interface Drawable {
    void draw();
}

interface Printable {
    void print();
}

class Document implements Drawable, Printable {
    @Override
    public void draw() { /* ... */ }

    @Override
    public void print() { /* ... */ }
}

// ABSTRACT CLASS approach
abstract class Shape {
    protected String color;  // Instance variable

    Shape(String color) {    // Constructor
        this.color = color;
    }

    abstract double getArea();  // Abstract

    void displayColor() {       // Concrete
        System.out.println("Color: " + color);
    }
}

class Circle extends Shape {
    private double radius;

    Circle(String color, double radius) {
        super(color);  // Call parent constructor
        this.radius = radius;
```

```
    }

    @Override
    double getArea() {
        return Math.PI * radius * radius;
    }
}
```

7. Complete Example: Drawable & Printable

```
// ═══════════════════════════════════════════════════════
// INTERFACES
// ═══════════════════════════════════════════════════════

interface Drawable {
    // Constants
    String DEFAULT_COLOR = "Black";
    int DEFAULT_STROKE_WIDTH = 1;

    // Abstract methods
    void draw();
    void setColor(String color);
    String getColor();

    // Default method
    default void drawWithBorder() {
        System.out.println("┌──────────────────────────┐");
        draw();
        System.out.println("└──────────────────────────┘");
    }

    // Static utility method
    static void printDrawingHeader() {
        System.out.println("╔══════════════════════════════════╗");
        System.out.println("║       DRAWING CANVAS             ║");
        System.out.println("╚══════════════════════════════════╝");
    }
}

interface Printable {
    // Constants
    String DEFAULT_PAPER_SIZE = "A4";

    // Abstract methods
    void print();
    int getPageCount();

    // Default method
```

```
    default void printMultipleCopies(int copies) {
        for (int i = 1; i <= copies; i++) {
            System.out.println("--- Copy " + i + " of " + copies + " ---");
            print();
        }
    }

    default void printPreview() {
        System.out.println("=== PRINT PREVIEW ===");
        System.out.println("Pages: " + getPageCount());
        System.out.println("Paper: " + DEFAULT_PAPER_SIZE);
        System.out.println("====================");
    }

    // Static method
    static void checkPrinterStatus() {
        System.out.println("Printer Status: Online ✓");
    }
}

interface Resizable {
    void resize(double factor);
    double getSize();
}

// ============================================================
// IMPLEMENTING CLASSES
// ============================================================

class Circle implements Drawable, Printable, Resizable {
    private String color;
    private double radius;

    Circle(double radius) {
        this.radius = radius;
        this.color = Drawable.DEFAULT_COLOR;
    }

    Circle(double radius, String color) {
        this.radius = radius;
```

```
        this.color = color;
    }

    // Drawable implementation
    @Override
    public void draw() {
        System.out.println("Drawing " + color + " circle with radius " + radius);
        System.out.println("    ****");
        System.out.println("  *      *");
        System.out.println(" *        *");
        System.out.println("  *      *");
        System.out.println("    ****");
    }

    @Override
    public void setColor(String color) {
        this.color = color;
    }

    @Override
    public String getColor() {
        return color;
    }

    // Printable implementation
    @Override
    public void print() {
        System.out.println("Printing Circle: radius=" + radius + ", color=" + color);
        System.out.println("Area: " + getArea());
    }

    @Override
    public int getPageCount() {
        return 1;
    }

    // Resizable implementation
    @Override
    public void resize(double factor) {
        radius *= factor;
```

```
        System.out.println("Circle resized to radius: " + radius);
    }

    @Override
    public double getSize() {
        return radius;
    }

    // Own method
    public double getArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle implements Drawable, Printable, Resizable {
    private String color;
    private double width;
    private double height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
        this.color = Drawable.DEFAULT_COLOR;
    }

    Rectangle(double width, double height, String color) {
        this.width = width;
        this.height = height;
        this.color = color;
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + color + " rectangle " + width + "x" + height);
        System.out.println("┌──────────┐");
        System.out.println("│          │");
        System.out.println("│          │");
        System.out.println("└──────────┘");
    }
```

```
    @Override
    public void setColor(String color) {
        this.color = color;
    }

    @Override
    public String getColor() {
        return color;
    }

    @Override
    public void print() {
        System.out.println("Printing Rectangle: " + width + "x" + height + ", color=" + color)
        System.out.println("Area: " + getArea());
    }

    @Override
    public int getPageCount() {
        return 1;
    }

    @Override
    public void resize(double factor) {
        width *= factor;
        height *= factor;
        System.out.println("Rectangle resized to: " + width + "x" + height);
    }

    @Override
    public double getSize() {
        return width * height;
    }

    public double getArea() {
        return width * height;
    }
}

class Document implements Printable {
    private String title;
```

```
    private String content;
    private int pages;

    Document(String title, String content, int pages) {
        this.title = title;
        this.content = content;
        this.pages = pages;
    }

    @Override
    public void print() {
        System.out.println("╔════════════════════════════════════╗");
        System.out.println("║  DOCUMENT: " + title);
        System.out.println("╠════════════════════════════════════╣");
        System.out.println("║  " + content);
        System.out.println("╚════════════════════════════════════╝");
    }

    @Override
    public int getPageCount() {
        return pages;
    }

    // Override default method
    @Override
    public void printPreview() {
        System.out.println("=== DOCUMENT PREVIEW ===");
        System.out.println("Title: " + title);
        System.out.println("Pages: " + pages);
        System.out.println("Content: " + content.substring(0, Math.min(50, content.length())) +
        System.out.println("========================");
    }
}

// ═══════════════════════════════════════════════════════
// MAIN CLASS - DEMONSTRATION
// ═══════════════════════════════════════════════════════

public class Main {
    public static void main(String[] args) {
```

```
        // ================================================
        // Using Interface Constants
        // ================================================
        System.out.println("Default Color: " + Drawable.DEFAULT_COLOR);
        System.out.println("Default Paper: " + Printable.DEFAULT_PAPER_SIZE);
        System.out.println();

        // ================================================
        // Using Static Methods
        // ================================================
        Drawable.printDrawingHeader();
        Printable.checkPrinterStatus();
        System.out.println();

        // ================================================
        // Creating Objects
        // ================================================
        Circle circle = new Circle(5, "Red");
        Rectangle rectangle = new Rectangle(10, 5, "Blue");
        Document document = new Document("Report", "This is the annual report content...", 10)

        // ================================================
        // Polymorphism with Drawable Interface
        // ================================================
        System.out.println("\n=== DRAWABLE DEMO ===\n");

        Drawable[] drawables = { circle, rectangle };

        for (Drawable d : drawables) {
            d.draw();
            d.drawWithBorder();  // Default method
            System.out.println();
        }

        // ================================================
        // Polymorphism with Printable Interface
        // ================================================
        System.out.println("\n=== PRINTABLE DEMO ===\n");
```

```
        Printable[] printables = { circle, rectangle, document };

        for (Printable p : printables) {
            p.printPreview();
            p.print();
            System.out.println();
        }

        // ═══════════════════════════════════════════════
        // Polymorphism with Resizable Interface
        // ═══════════════════════════════════════════════
        System.out.println("\n=== RESIZABLE DEMO ===\n");

        Resizable[] resizables = { circle, rectangle };

        System.out.println("Before resize:");
        for (Resizable r : resizables) {
            System.out.println("Size: " + r.getSize());
        }

        System.out.println("\nResizing by factor 2:");
        for (Resizable r : resizables) {
            r.resize(2);
        }

        System.out.println("\nAfter resize:");
        for (Resizable r : resizables) {
            System.out.println("Size: " + r.getSize());
        }

        // ═══════════════════════════════════════════════
        // Using Default Methods
        // ═══════════════════════════════════════════════
        System.out.println("\n=== DEFAULT METHODS DEMO ===\n");

        document.printMultipleCopies(2);  // Default method

        // ═══════════════════════════════════════════════
        // Instance of Check
        // ═══════════════════════════════════════════════
```

```
        System.out.println("\n=== INSTANCEOF DEMO ===\n");

        Object[] objects = { circle, rectangle, document, "Hello" };

        for (Object obj : objects) {
            System.out.println(obj.getClass().getSimpleName() + ":");
            System.out.println("  Drawable? " + (obj instanceof Drawable));
            System.out.println("  Printable? " + (obj instanceof Printable));
            System.out.println("  Resizable? " + (obj instanceof Resizable));
        }
    }
}
```

Output

```
Default Color: Black
Default Paper: A4


╔═══════════════════════════════════════╗
║       DRAWING CANVAS          ║
╚═══════════════════════════════════════╝
Printer Status: Online ✓

=== DRAWABLE DEMO ===

Drawing Red circle with radius 5.0
    ****
  *      *
 *        *
  *      *
    ****
┌──────────────────────────┐
Drawing Red circle with radius 5.0
    ****
  *      *
 *        *
  *      *
    ****
└──────────────────────────┘


Drawing Blue rectangle 10.0x5.0
┌──────────┐
│          │
│          │
└──────────┘

┌──────────────────────────┐
Drawing Blue rectangle 10.0x5.0
┌──────────┐
│          │
│          │
└──────────┘
└──────────────────────────┘
```

```
=== PRINTABLE DEMO ===

=== PRINT PREVIEW ===
Pages: 1
Paper: A4
====================
Printing Circle: radius=5.0, color=Red
Area: 78.53981633974483

... (continues)
```

---

8. Key Points to Remember

Interface Basics

1. Interface defines a **contract** — what methods must exist
2. All methods are `public abstract` by default
3. All variables are `public static final` (constants)
4. Use `implements` keyword to implement
5. A class can implement **multiple** interfaces

Method Types

6. **Abstract methods** — no body, must be implemented
7. **Default methods** (Java 8+) — have body, can be overridden
8. **Static methods** (Java 8+) — belong to interface, not inherited
9. **Private methods** (Java 9+) — helper methods for default/static

Multiple Inheritance

10. Interfaces solve the diamond problem

11. Interface can extend multiple interfaces

12. Class can extend ONE class but implement MANY interfaces

Interface vs Abstract Class

13. Interface = **CAN-DO** (capability)

14. Abstract Class = **IS-A** (inheritance)

15. Use interface for loose coupling and contracts

16. Use abstract class for shared code and state

'; triggerDownload(docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); } function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); // Force light mode for PDF (dark backgrounds print poorly) const wasDark = document.documentElement.classList.contains('dark'); if (wasDark) document.documentElement.classList.remove('dark'); const element = document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename: docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale: 1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' }, pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if (wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme: dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !== 'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i < mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const container = document.createElement('div'); container.className = 'mermaid-container'; container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark; document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */ window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark);

updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} } });